

Design e Implementação de uma Placa de Som USB Dual-Core com Equalizador Paramétrico e Análise Espectral FFT via Node-RED

Professor Orientador: Ricardo Andrade

Igor Sadowski
Departamento de Eletrônica Industrial
Universidade do Minho
Guimarães, Portugal
a108957@uminho.pt

Nádia Silva
Departamento de Eletrônica Industrial
Universidade do Minho
Guimarães, Portugal
a107054@uminho.pt

Lara Fernandes
Departamento de Eletrônica Industrial
Universidade do Minho
Guimarães, Portugal
a107019@uminho.pt

Vítor Araújo
Departamento de Eletrônica Industrial
Universidade do Minho
Guimarães, Portugal
a107100@uminho.pt

Resumo—Este artigo apresenta o desenvolvimento de uma placa de som USB baseada no microcontrolador dual-core STM32H755ZI-Q. O objetivo do projeto foi criar um protótipo capaz de receber áudio digital de um computador, processá-lo em tempo real e apresentar simultaneamente uma visualização espectral do sinal. O sistema implementa o middleware USB Device com o protocolo USB Audio Class 1.0 em modo assíncrono para recepção de áudio a 48 kHz. Na arquitetura desenvolvida, o core Cortex-M7 gere a comunicação USB, aplica filtros digitais IIR em cascata para implementar o equalizador paramétrico e envia o áudio processado para um DAC externo através do periférico SAI. Em paralelo, o core Cortex-M4 calcula uma FFT de 4096 pontos, aplica uma janela de Hann e extrai 128 bandas logarítmicas enviadas por USART3 para um dashboard Node-RED. O protótipo foi reconhecido como placa de som USB externa, reproduziu áudio através do DAC, aplicou perfis de equalização e apresentou o espectro em tempo real. O trabalho demonstra a utilidade de arquiteturas dual-core em sistemas embedded de áudio, permitindo separar a cadeia crítica de reprodução das tarefas de análise e visualização.

Palavras-chave—STM32H7, USB Audio Class, processamento digital de sinal, dual-core, CMSIS-DSP, FFT, Node-RED, sistemas embedded

I. INTRODUÇÃO E DEFINIÇÃO DO PROBLEMA

A reprodução de áudio digital de alta fidelidade a partir de um computador para um periférico dedicado exige um fluxo de dados contínuo, estável e determinístico. No caso de uma placa de som USB externa, um dos principais desafios está relacionado com a sincronização entre relógios de hardware independentes, nomeadamente o relógio do computador e o relógio do microcontrolador, e também a capacidade de processamento necessária para aplicar efeitos de áudio em tempo real sem provocar underflows ou overflows nos buffers.

Neste projeto, o objetivo consistiu em desenvolver um dispositivo USB capaz de funcionar como placa de som externa, recebendo áudio digital de um computador, processando esse sinal em tempo real e enviando-o para um DAC externo. Para além da reprodução de áudio, o sistema implementa um equalizador paramétrico de 10 bandas e uma análise espectral em tempo real, permitindo visualizar o

conteúdo em frequência do sinal através de uma interface Node-RED.

A execução simultânea da recepção USB, do processamento DSP, da saída de áudio e da análise FFT representa uma carga computacional significativa. Deste modo, se todas estas tarefas fossem concentradas num único processador, poderiam ocorrer atrasos no processamento e instabilidade no fluxo de áudio. Para responder a este problema, foi utilizada a arquitetura dual-core disponível no STM32H755ZI-Q, separando as tarefas críticas de áudio no Cortex-M7 e a análise espectral no Cortex-M4 [1], [2]. Desta forma, o Cortex-M7 fica responsável por gerir a comunicação USB, aplicar os filtros digitais do equalizador e enviar o áudio para o DAC, enquanto o Cortex-M4 calcula a Transformada Rápida de Fourier e envia as bandas espectrais para o Node-RED.

A. Requisitos de Engenharia

Para garantir a viabilidade do sistema, foram definidos vários requisitos de engenharia. Em primeiro lugar, o dispositivo deveria implementar comunicação USB assíncrona através do protocolo USB Audio Class 1.0, permitindo que o computador reconhecesse o protótipo como uma placa de som USB externa [8]. Devido à natureza assíncrona do sistema, tornou-se necessário incluir um mecanismo de feedback capaz de compensar diferenças entre o ritmo de envio do computador e o ritmo de consumo do dispositivo.

Outro requisito fundamental foi a realização do processamento digital de sinal no Cortex-M7. Este core ficou responsável pela aplicação de um equalizador paramétrico de 10 bandas em tempo real, baseado em filtros digitais IIR em cascata, permitindo alternar entre diferentes perfis acústicos, como Flat, Radio e Bass Boost [11], [12].

Foi também necessário garantir a conversão digital-analógica do sinal de áudio. Para isso, o áudio processado é enviado para um DAC externo PCM5102 [17] através da interface SAI, em formato I2S, permitindo obter um sinal analógico audível à saída do sistema.

Em paralelo, o sistema deveria realizar análise espectral no Cortex-M4. Este core ficou responsável pelo cálculo da Transformada Rápida de Fourier a partir das amostras de

áudio, agrupando os resultados em bandas logarítmicas para representar visualmente a distribuição de energia pelas frequências audíveis [13].

Por fim, foi definido como requisito a visualização externa dos dados espectrais. As amplitudes das bandas calculadas são transmitidas através da USART3 para um dashboard Node-RED, evitando sobrecarregar o microcontrolador com tarefas de renderização gráfica local [16].

II. METODOLOGIA E ARQUITETURA DO SISTEMA

A. Periféricos e Middlewares Utilizados

A estabilidade do sistema depende de uma árvore de relógios rigorosamente configurada a partir do cristal externo HSE de 8 MHz, permitindo gerar as frequências necessárias para a comunicação USB, para o periférico SAI e para os restantes módulos do sistema [1], [3].

A Tabela I apresenta os principais periféricos e middlewares utilizados na implementação do sistema.

TABELA I. CONFIGURAÇÃO DOS PERIFÉRICOS E MIDDLEWARES UTILIZADOS

Recurso / Middleware	Núcleo	Configuração / Frequência	Função
USB_OTG_FS	M7	Device Mode (48 MHz via PLL3)	Recurso de hardware físico para ligação ao barramento USB Full-Speed.
USB_DEVICE	M7	Middleware: UAC 1.0	Stack de software geradora dos descritores e gestão dos endpoints áudio.
SAI1	M7	Master TX, I2S (12.288 MHz via PLL2)	Transmissão rigorosa a 48 kHz exatos para o DAC PCM5102.
DMA1	M7	Circular, Mem-to-Periph	Alimentação circular automática do periférico SAI1 (descarregando o CPU).
USART3	M4	Assíncrona, 115200 bps	Transmissão das amplitudes da FFT processadas para a interface Node-RED.
SRAM4	Ambos	Endereço 0x38000000	Memória partilhada (IPC) para Ping-Pong buffers de áudio.

B. Stack USB, Descritores e Mecanismo de Feedback

A comunicação USB constitui uma das partes mais críticas do sistema, uma vez que é responsável por garantir a receção contínua do fluxo de áudio enviado pelo computador. Para que o protótipo seja reconhecido como uma placa de som externa e consiga manter uma reprodução estável, foi necessário configurar corretamente a stack USB, os descritores do dispositivo e o mecanismo de feedback assíncrono.

Numa primeira fase, a interface com o computador host é gerida pelo middleware USB Device da STMicroelectronics, configurado para o protocolo USB Audio Class 1.0 (UAC 1.0) [8]. O funcionamento desta stack baseia-se em descritores USB, que informam o

sistema operativo sobre a topologia e as capacidades do dispositivo. Assim, os Device e Configuration Descriptors definem a identificação geral do dispositivo, apresentado como “STM32 DSP/DAC 10-PEQ”. Por sua vez, os Interface Descriptors dividem-se em Audio Control e Audio Streaming, sendo esta última interface responsável por definir as características dos pacotes de áudio PCM, neste caso a 48 kHz, estéreo e 24 bits empacotados em 3 bytes por canal [10]. Por fim, os Endpoint Descriptors definem os canais físicos de dados, sendo utilizados dois endpoints isócronos: o endpoint 0x01 (OUT), para receção do áudio, e o endpoint 0x81 (IN), para envio do feedback.

Depois de o dispositivo ser reconhecido pelo computador, o principal desafio passa a ser a sincronização do fluxo de áudio. Na transferência isócrona, o computador envia dados a cada 1 ms, através de pacotes SOF (Start Of Frame) [6]. No entanto, o relógio local do STM32, gerado a partir do cristal HSE e das PLL, não é exatamente igual ao relógio do computador. Esta diferença, juntamente com o jitter do barramento USB Full-Speed e a latência do sistema operativo, pode provocar flutuações no ritmo de chegada dos dados. Consequentemente, sem um mecanismo de sincronização, o buffer de áudio poderia esvaziar-se, originando underrun, ou encher em excesso, originando overflow, ambos perceptíveis como estalidos ou falhas na reprodução.

Para evitar este problema, o endpoint de dados foi configurado como assíncrono, permitindo que o dispositivo controle indiretamente o ritmo de envio do host através de um endpoint de feedback [8]. Esta lógica é implementada na função `USB_AUDIO_Feedback_SOF()`, chamada a cada pacote SOF, ou seja, a cada 1 ms. No entanto, a correção não é aplicada em todos os SOF: a cada 16 iterações, correspondentes a 16 ms, o Cortex-M7 calcula a diferença entre o ponteiro de escrita atual, `write_ptr`, e a posição de leitura do DMA, obtida através de `__HAL_DMA_GET_COUNTER()`. O objetivo do algoritmo é manter esta distância próxima do centro do buffer circular `SaiPlaybackBuffer`.

Com base neste erro, o valor de feedback, `fback_val`, é ajustado iterativamente. No protocolo USB Audio Class, este valor é enviado no formato 10.14, com 10 bits para a parte inteira e 14 bits para a parte fracionária [8]. Para uma taxa de 48 kHz, o valor nominal corresponde a 48 amostras por milissegundo, ou seja, $48 \times 2^{14} = 786432$, equivalente a 0x0C0000 em hexadecimal. Deste modo, se o DMA estiver a consumir dados mais rapidamente do que o computador os envia, o valor de feedback aumenta ligeiramente, indicando ao driver do sistema operativo que deve enviar mais amostras. Pelo contrário, se o buffer estiver a encher em excesso, o valor é reduzido. Para evitar oscilações, o algoritmo utiliza diferentes limiares de correção, onde erros superiores a 1024 amostras originam ajustes mais fortes, erros entre 128 e 1024 amostras originam ajustes intermédios, e erros entre 32 e 128 amostras originam ajustes finos.

Este processo decorre continuamente durante a reprodução. Após o cálculo, o valor final de feedback, com 24 bits, é convertido em 3 bytes e armazenado em `audio_feedback_buf`. Em seguida, o envio físico para o host é realizado através de `HAL_PCD_EP_Transmit()`, apenas quando o endpoint de feedback não se encontra ocupado,

verificado através de `hpcd->IN_ep[1].xfer_len == 0`, evitando sobreposição de transmissões [3].

Além do mecanismo de feedback, a estabilidade do sistema também depende da dimensão do buffer de recepção isócrono. No ficheiro `usbd_audio.h`, o parâmetro `AUDIO_OUT_PACKET` define o tamanho máximo de cada pacote recebido do host, fixado neste projeto em 300 bytes. Este valor excede ligeiramente os 288 bytes necessários para uma trama de 48 kHz, 2 canais e 24 bits, dado que cada milissegundo corresponde a 48 amostras estéreo de 6 bytes. Já o parâmetro `AUDIO_OUT_PACKET_NUM` define o número de pacotes armazenados no buffer circular interno, sendo usado o valor de 200 pacotes. Assim, o tamanho total do buffer, `AUDIO_TOTAL_BUF_SIZE`, é de 60000 bytes. Este buffer encontra-se associado ao array `audio->buffer`, pertencente à estrutura `USBD_AUDIO_HandleTypeDef`, permitindo absorver pequenas variações no ritmo de chegada dos dados e complementando o mecanismo de feedback assíncrono.

Em conjunto, estes mecanismos asseguram a sincronização entre o computador host e o dispositivo, permitindo uma reprodução de áudio contínua e sem artefactos audíveis relevantes.

C. Motor DSP, MPU e Coerência de Cache (Cortex-M7)

Após a recepção e sincronização dos dados através da interface USB, o processamento principal do sistema é executado pelo núcleo Cortex-M7. Este núcleo é responsável pela gestão dos buffers de áudio, pela execução do equalizador paramétrico, pela preparação dos dados para reprodução através do DAC e pela comunicação com o Cortex-M4 para análise espectral [1], [2].

De forma a garantir uma reprodução contínua e sem interrupções, foi implementada uma arquitetura baseada em buffers circulares, processamento em ponto flutuante e mecanismos avançados de coerência de memória. As secções seguintes descrevem detalhadamente cada uma destas etapas.

1. Recepção e Gestão dos Dados USB

A recepção dos dados USB é gerida internamente pelo middleware `usbd_audio.c`, que mantém um buffer circular com os ponteiros `wr_ptr`, para escrita, e `rd_ptr`, para leitura, além da flag `rd_enable`, que sinaliza quando o buffer está suficientemente preenchido para iniciar a reprodução. Sempre que um pacote isócrono chega ao endpoint `0x01`, a função `USBD_AUDIO_DataOut()` é chamada por interrupção [3], [8].

Numa primeira fase, esta função obtém o tamanho real do pacote recebido através de `USBD_LL_GetRxDataSize()`. De seguida, invoca a callback `PeriodicTC()`, mapeada para `AUDIO_PeriodicTC_FS()`, passando o endereço do sub-bloco atual do buffer interno, `&audio->buffer[audio->wr_ptr]`, e o respetivo tamanho do pacote. Esta callback chama então `SAMPLE_PROCESSING()`, responsável por processar as amostras recebidas e escrevê-las no `SaiPlaybackBuffer`.

Depois deste processamento, o ponteiro `wr_ptr` é incrementado de acordo com o tamanho do pacote recebido. Caso atinja o limite do buffer, é reposto a zero, garantindo o comportamento circular. Quando `wr_ptr` atinge metade do buffer, `AUDIO_TOTAL_BUF_SIZE / 2`, a flag `rd_enable` é

ativada, permitindo ao middleware considerar que já existem dados suficientes para iniciar a libertação de áudio. Por fim, `USBD_LL_PrepareReceive()` é chamada para preparar o endpoint para o próximo pacote, reiniciando o ciclo de recepção.

Este esquema de double buffering circular no interior do middleware isola a camada de aplicação dos detalhes temporais do barramento USB. Assim, `SAMPLE_PROCESSING()` recebe sempre um bloco contíguo e completo de dados PCM no contexto da recepção completa do endpoint de áudio, sempre que chega um novo pacote USB ao endpoint `0x01`.

No presente projeto, contudo, a flag `rd_enable` e o ponteiro `audio->rd_ptr` do middleware USB não são utilizados para controlar diretamente a reprodução. Em vez de o DMA do SAI ler diretamente de `audio->buffer`, como previsto no middleware original, o sistema interceta os dados na callback `PeriodicTC()`, aplica o processamento DSP e escreve os resultados no `SaiPlaybackBuffer`, utilizando o seu próprio índice `write_ptr`, definido em `usbd_audio_if.c`. Desta forma, a gestão do buffer de saída fica a cargo deste índice, tornando os mecanismos de leitura do middleware, `audio->rd_ptr` e `rd_enable`, dispensáveis para o pipeline final de áudio. O ponteiro `audio->wr_ptr` continua associado à recepção intermédia no buffer interno, mas a reprodução efetiva passa a depender do `SaiPlaybackBuffer`, onde são colocadas as amostras já processadas.

2. Extração e Conversão de Dados (24-bit para Float32)

O fluxo USB fornece amostras PCM estéreo de 24 bits, empacotadas em 3 bytes consecutivos por canal, o que corresponde a 6 bytes por amostra estéreo L+R [10]. Antes de aplicar o processamento digital, é necessário reconstruir cada amostra para um valor inteiro com sinal.

Para isso, a função percorre o buffer de origem e utiliza operações de deslocamento de bits para combinar os três bytes de cada canal num valor de 24 bits:

```
int32_t left_24 = (sample[0]) | (sample[1] << 8) |  
(sample[2] << 16);
```

Como o sinal de áudio pode assumir valores positivos e negativos, é aplicada extensão de sinal sempre que o bit mais significativo dos 24 bits se encontra ativo:

```
if (left_24 & 0x00800000) left_24 |= 0xFF000000;
```

Após esta reconstrução, o valor inteiro é convertido para `float32_t`. Para isso, divide-se o valor por 2^{23} , isto é, 8388608.0f, normalizando a amostra para o intervalo [-1.0, 1.0]. Esta conversão permite que o DSP e a FFT operem em ponto flutuante, com maior precisão matemática durante o processamento.

3. Processamento DSP e Equalização Paramétrica

Após a conversão das amostras para o formato `float32_t`, o sinal normalizado é enviado para a função `DSP_Process_Sample()`. Esta função utiliza a biblioteca CMSIS-DSP para aplicar os filtros de equalização paramétrica em cascata, através da função `arm_biquad_cascade_df1_f32()` [11], [12].

No interior de `DSP_Process_Sample()`, o sinal passa primeiro pela cascata de filtros IIR, que pode introduzir ganhos superiores a 1.0 em determinadas bandas de frequência. Por esse motivo, é aplicada uma limitação do sinal a ± 0.99 , evitando overflow numérico nas etapas seguintes. Depois desta limitação, é aplicada uma atenuação global de 20%, através das operações $*L *= 0.80f$ e $*R *= 0.80f$. Esta ordem garante que os picos de ganho introduzidos pelo equalizador são controlados antes da atenuação final, maximizando a relação sinal-ruído interna.

Após o processamento DSP, a amostra em ponto flutuante, no intervalo $[-1.0, 1.0]$, é convertida para inteiro de 32 bits com sinal. O fator de escala utilizado é $2^{31} - 1$, correspondente a 2147483647, que representa o valor máximo positivo de um inteiro de 32 bits com sinal. Desta forma, a gama completa do formato inteiro é aproveitada, maximizando a resolução da conversão.

O valor inteiro resultante é inserido no `SaiPlaybackBuffer`, um buffer circular gerido exclusivamente pelo índice `write_ptr`. O DMA lê continuamente este buffer em modo circular, alimentando o periférico SAI, que transmite os 32 bits para o DAC PCM5102 [17]. Em paralelo, a rotina calcula a média mono, dada por $(sL + sR) * 0.5f$, e armazena-a num dos dois buffers da estrutura `SHARED_DATA`, localizada na SRAM4. A alternância entre `audio_buffer_0` e `audio_buffer_1` ocorre a cada 4096 amostras, sendo depois ativada a flag `data_ready` para notificar o Cortex-M4 de que existe um novo bloco disponível para análise espectral.

a) Implementação do PEQ

A implementação prática da equalização no firmware não se baseia no cálculo matemático completo em tempo real, mas sim na correta organização das estruturas de dados, gestão de coeficientes e integração com o pipeline de interrupções. O código foi estruturado para permitir a comutação de perfis e o processamento das amostras com o mínimo de overhead, sem bloquear a thread crítica de áudio.

i. Estruturas de Dados do Filtro

Para cada canal, esquerdo e direito, o programa declara uma variável de instância do tipo `arm_biquad_cascd_df1_inst_f32`, definida pela CMSIS-DSP [12]. Esta estrutura contém dois campos fundamentais: `pState`, que aponta para um array de estado em floating-point onde são guardados os valores históricos das amostras anteriores, necessários para a realimentação do filtro; e `pCoeffs`, que aponta para o array de coeficientes do filtro, com 5 coeficientes por banda e um total de 50 valores para as 10 bandas do equalizador.

A inicialização é feita em `DSP_Init()`, que chama `DSP_Set_Mode(MODE_FLAT)` para configurar o primeiro perfil de equalização.

ii. Cálculo dos Coeficientes e Comutação de Perfis

O sistema suporta três perfis acústicos: Flat, Radio e Bass Boost. Cada perfil é definido por uma tabela de configuração do tipo `EQ_BandConfig`, nomeadamente `profile_flat`, `profile_radio` e `profile_bass`. Estas tabelas contêm apenas parâmetros de alto nível, como a frequência central, o fator de qualidade Q, o ganho em dB e o tipo de filtro, que pode ser Peaking, Low-Shelf ou High-Shelf.

Quando o utilizador pressiona o botão físico ligado ao GPIO PC13, a rotina de interrupção `HAL_GPIO_EXTI_Callback()` incrementa o índice do perfil e ativa a flag `dsp_update_req`. No loop principal do Cortex-M7, esta flag é detetada e é chamada a função `DSP_Set_Mode()` com o novo modo selecionado.

Dentro de `DSP_Set_Mode()`, é obtido o ponteiro para a tabela de configuração do perfil ativo. Em seguida, para cada uma das 10 bandas, a função `CalcBiquad()` calcula os 5 coeficientes do filtro Biquad utilizando as fórmulas do Audio EQ Cookbook [14]. Os coeficientes calculados são armazenados nos arrays estáticos `coeffs_L` e `coeffs_R`. Por fim, as estruturas dos filtros são reinicializadas com `arm_biquad_cascade_df1_init_f32()`, associando os novos coeficientes e limpando o estado dos filtros através de `memset(state_L, 0, sizeof(state_L))` e `memset(state_R, 0, sizeof(state_R))`.

4. Gestão de Memória Avançada (MPU e D-Cache)

O STM32H7 possui uma Data Cache L1, que permite aumentar significativamente o desempenho do Cortex-M7. No entanto, a existência de cache pode gerar problemas de coerência quando há partilha de dados entre o Cortex-M7, o Cortex-M4 e periféricos que acedem diretamente à memória, como o DMA [1], [2].

Para a comunicação entre os dois núcleos, foi utilizada a SRAM4, localizada no endereço `0x38000000`. Através da Unidade de Proteção de Memória (MPU), esta região foi configurada como Non-Cacheable e Non-Bufferable. Desta forma, a mistura mono gravada pelo Cortex-M7 é escrita diretamente na RAM física, garantindo que o Cortex-M4 lê sempre dados atualizados quando a flag `data_ready` é ativada [1], [2].

Por outro lado, o `SaiPlaybackBuffer` é lido continuamente pelo DMA do periférico SAI. Como o DMA acede diretamente à memória física, enquanto o Cortex-M7 pode manter dados recentes apenas na cache, torna-se necessário descarregar explicitamente para a memória as regiões modificadas. Para isso, após a escrita das amostras no `SaiPlaybackBuffer`, a função `SAMPLE_PROCESSING()` utiliza `SCB_CleanDCache_by_Addr()`, garantindo que o DMA lê os dados mais recentes [3].

A lógica implementada otimiza esta operação ao limpar apenas as regiões efetivamente escritas, evitando o flush total do buffer e reduzindo o overhead. Esta solução foi validada empiricamente, uma vez que a remoção da limpeza da cache resultou em artefactos audíveis, como estalidos, confirmando a necessidade deste mecanismo para garantir a integridade do áudio.

Em conjunto, estes mecanismos permitem ao Cortex-M7 processar o fluxo de áudio em tempo real, assegurando simultaneamente a correta alimentação do DAC, a partilha eficiente de dados com o Cortex-M4 e a integridade da informação armazenada em memória.

D. Análise Espectral no Cortex-M4: FFT, Leakage e Mapeamento Logarítmico

O Cortex-M4 aguarda a ativação da flag `data_ready` e atua como coprocessador matemático, ficando responsável pelo cálculo da Transformada Rápida de Fourier (FFT) e pela preparação dos dados usados na visualização espectral.

1. Cálculo de Bins e Janela de Hann

Ao processar blocos de 4096 amostras a uma taxa de amostragem de 48 kHz, a resolução em frequência de cada bin da FFT é de aproximadamente 11.72 Hz por bin. Para reduzir o fenómeno de fuga espectral, ou seja, o espalhamento de energia para bins adjacentes causado pelo corte temporal abrupto do bloco de áudio, o sinal é previamente multiplicado por uma janela de Hann [15]. Esta janela suaviza os extremos do bloco, reduzindo descontinuidades artificiais e concentrando a energia nas frequências dominantes do sinal. De seguida, a função `arm_rfft_fast_f32()`, da biblioteca CMSIS-DSP, calcula a FFT real do bloco de áudio, sendo depois extraídas as 2048 magnitudes úteis até à frequência de Nyquist, descartando o bin 0, correspondente à componente DC [11], [13].

2. Mapeamento Logarítmico e Interpolação Sub-Bin

A distribuição linear dos bins da FFT não reflete diretamente a perceção auditiva humana, que é aproximadamente logarítmica. Para aproximar a visualização da forma como o ouvido humano percebe as frequências, o algoritmo agrupa os bins em 128 bandas logarítmicas, com limites distribuídos entre 20 Hz e 20 kHz segundo uma progressão geométrica.

Nas frequências graves, a largura de algumas bandas logarítmicas pode ser inferior à resolução de um único bin. Uma atribuição direta a bins inteiros poderia fazer com que várias barras partilhassem o mesmo valor, criando uma visualização pouco fluida. Para evitar este efeito, o algoritmo calcula o centro fracionário da banda e aplica interpolação linear entre dois bins adjacentes quando a largura da banda é inferior ou igual a 1 bin. O valor da banda é obtido através da média ponderada:

$$\text{pico} = (\text{mag}[\text{idx}] * (1.0 - \text{frac})) + (\text{mag}[\text{idx} + 1] * \text{frac})$$

Nas bandas de frequência mais elevada, onde cada banda abrange vários bins, a magnitude é obtida através da deteção do pico máximo dentro do intervalo correspondente.

3. Compensação, Normalização e Modelação Visual

As magnitudes extraídas são convertidas para a escala de decibéis. Como a energia natural da música tende a diminuir nas frequências mais altas, foi introduzido um ganho de compensação progressivo, dado por $\text{db} += b * 0.20$, onde db representa a magnitude em decibéis e b corresponde ao índice da banda. Esta compensação permite equilibrar visualmente a dinâmica dos agudos em relação aos graves.

Posteriormente, o valor é normalizado numa gama dinâmica de 80 dB, ignorando componentes de baixa amplitude abaixo de -20 dB. Para tornar os picos mais definidos na visualização, aplica-se uma exponenciação de afinamento através de $\text{powf}(\text{norm}, 1.4f)$. Desta forma, valores próximos do máximo sofrem pouca alteração, enquanto valores intermédios são atenuados, resultando em barras mais nítidas.

Finalmente, os dados são enviados via USART3 para o dashboard Node-RED [16]. No loop principal do Cortex-M4, após o envio de cada pacote, realizado a cada 50 ms, é aplicado um decaimento a todas as bandas, multiplicando

cada valor por 0.85. Este efeito cria uma transição mais suave entre atualizações sucessivas, evitando que as barras do dashboard caiam abruptamente para zero quando a amplitude do sinal diminui.

E. Gestão de Prioridades de Interrupção (NVIC)

Para garantir o determinismo e reduzir a ocorrência de glitches no áudio, a configuração das prioridades das interrupções no Nested Vectored Interrupt Controller (NVIC) foi cuidadosamente hierarquizada [1], [3]. A Tabela II resume as prioridades atribuídas.

TABELA II. CONFIGURAÇÃO DAS PRIORIDADES DE INTERRUPÇÃO DO NVIC

Periférico / Interrupção	Prioridade (Preempção)	Subprioridade	Justificação
OTG_FS_IRQn (USB)	0	0	A receção de pacotes isócronos e o envio de feedback são a origem de todo o áudio; qualquer atraso nesta interrupção causa underrun no host.
DMA1_Stream0_IRQn (SAI)	1	0	O DMA do SAI alimenta o DAC; a sua interrupção (Half/Full) é necessária para manter a sincronização com o middleware USB.
SAI1_IRQn	1	0	Interrupção geral do periférico SAI, partilhando a mesma prioridade do DMA.
EXTI15_10_IRQn (Botão)	0	0	Embora tenha prioridade máxima, a rotina de serviço é extremamente curta (apenas incrementa uma variável e ativa uma flag), não bloqueando as interrupções críticas.

A atribuição de prioridade mais alta ao USB, com prioridade 0, e prioridade intermédia ao DMA/SAI, com prioridade 1, assegura que os pacotes de áudio são recebidos e que o feedback é enviado pontualmente, mesmo quando o processamento DSP ou a comutação de perfil, realizada no loop principal, está a decorrer. A interrupção do botão, apesar de ter a mesma prioridade máxima que o USB, foi projetada para ser não bloqueante e extremamente curta, minimizando o seu impacto no fluxo de áudio.

III. INTERFACE GRÁFICA. ECRÃ LOCAL VS NODE-RED

Durante a fase de desenvolvimento do projeto, foi inicialmente considerada a utilização de um ecrã local LCD

TFT, baseado no controlador ST7789 e comunicado por SPI, gerido pelo Cortex-M4. No entanto, esta abordagem revelou limitações práticas ao nível da largura de banda e da carga de processamento necessária para atualizar continuamente a interface gráfica.

A atualização de 128 barras espectrais a uma taxa de aproximadamente 20 frames por segundo exigiria o envio frequente de comandos e dados gráficos através da interface SPI. Esta operação introduziria uma carga adicional significativa no Cortex-M4, podendo atrasar o processamento de novos blocos de áudio e a execução da FFT. Assim, a utilização de um ecrã local poderia comprometer a separação entre a análise espectral e a cadeia crítica de áudio.

Como alternativa, optou-se por transferir a renderização gráfica para o computador, utilizando a plataforma Node-RED [16]. Nesta solução, o Cortex-M4 converte as amplitudes das 128 bandas espectrais num pacote ASCII e transmite os dados através da USART3, configurada a 115200 bps. O Node-RED recebe a informação pela porta série e apresenta-a num dashboard, sob a forma de barras dinâmicas representativas do espectro do áudio.

Esta abordagem permite manter o microcontrolador focado no cálculo da FFT e na preparação dos dados, deixando a tarefa de renderização gráfica a cargo do computador. Como resultado, obteve-se uma interface visual mais flexível, sem acrescentar uma carga significativa de processamento ao STM32.

IV. RESULTADOS E TESTES

A arquitetura desenvolvida foi validada através de testes funcionais ao dispositivo USB, à reprodução de áudio, ao mecanismo de feedback e à visualização espectral.

A. Estimativa de Latência do Sistema e Estabilidade USB

O dispositivo foi enumerado como dispositivo de áudio nativo no sistema operativo Linux.

```
Every 1.0s: cat /proc/asound/card3/stream0
WELES-Sadowskiego STM32 DSP/DAC 10-PEQ at usb-0000:05:00.4-2, full speed : USB Audio

Playback:
Status: Running
Interface = 1
Altset = 1
Packet Size = 300
Momentary freq = 48130 Hz (0x30.2134)
Feedback Format = 10.14

Interface 1
Altset 1
Format: S24_3LE
Channels: 2
Endpoint: 0x01 (1 OUT) (ASYNC)
Rates: 48000
Bits: 24
Channel map: FL FR
Sync Endpoint: 0x81 (1 IN)
Sync EP Interface: 1
Sync EP Altset: 1
Implicit Feedback Mode: No
```

Fig. 1. Enumeração do dispositivo USB no sistema operativo. É possível observar o Manufacturer String “WELES-Sadowskiego”, o Product Name “STM32 DSP/DAC 10-PEQ”, os endpoints isócronos configurados, nomeadamente o Endpoint 0x01 para áudio OUT e o Endpoint 0x81 para feedback IN, e as restantes configurações do dispositivo.

A latência global do sistema, desde a ordem de reprodução no PC até à saída analógica no DAC, foi estimada teoricamente no intervalo de 60 a 90 ms. Este valor não resulta de uma medição física ou instrumental direta,

mas é deduzido a partir da configuração arquitetural do firmware.

Explicitamente, o código em `usbd_audio_if.c` define `SAI_BUFFER_MS = 100` e só inicia o DMA do SAI após receber `SAI_BUFFER_MS / 2 = 50` pacotes USB, ou seja, 50 ms de pré-preenchimento. Esta condição é implementada no firmware através de `if (++packets_received >= SAI_BUFFER_MS / 2)`. Tendo em conta que o barramento Full-Speed envia um pacote a cada milissegundo, este valor corresponde a um atraso configurado de 50 ms. A este valor somam-se as latências naturais impostas pelas stacks de áudio do sistema operativo do PC, tipicamente entre 15 e 30 ms.

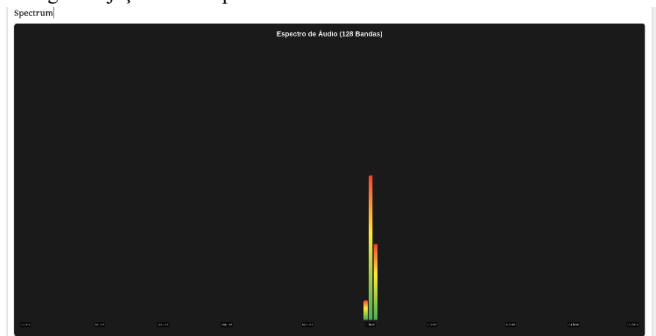
Para audição musical comum, este atraso é aceitável. O buffer total de 100 ms, aliado ao endpoint de feedback assíncrono, permite que o buffer se mantenha estável e evita a ocorrência de underflows e overflows. Durante os testes realizados, não foram observados artefactos auditivos relevantes, como clicks ou pops, associados à instabilidade do fluxo USB.

B. Visualização e Testes com Sons Puros

Para confirmar a eficácia da janela de Hann e a correta conversão dos bins da FFT, injetaram-se ondas sinusoidais puras no dispositivo. Os resultados obtidos estão em conformidade com a teoria do processamento digital de sinais, nomeadamente no que diz respeito à resolução em frequência da FFT e ao efeito da janela aplicada sobre a distribuição de energia pelos bins adjacentes [13], [15].

A FFT foi configurada com 4096 pontos a uma taxa de amostragem de 48 kHz, resultando numa resolução de aproximadamente 11.72 Hz por bin. Esta resolução fina permite uma análise espectral detalhada, mas também revela o comportamento discreto da transformada: frequências que não coincidem exatamente com o centro de um bin distribuem a sua energia pelos bins adjacentes.

Fig. 2. Injeção de tom puro a 1000 Hz.



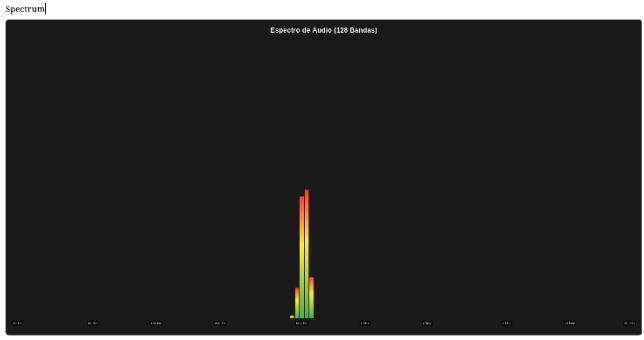


Fig. 3. Injeção de tom puro a 500 Hz.

1. Análise Matemática dos Testes com Tons Puros Baseada no Código

A resolução da FFT no sistema é dada por:

$$\Delta f = \frac{f_s}{N} = \frac{48000}{4096} = \frac{11.71875 \text{ Hz}}{\text{bin}} \quad (1)$$

Assim, o índice correspondente a uma frequência física pode ser obtido dividindo essa frequência pela resolução espectral da FFT.

a) Tom de 1 kHz

Considerando a resolução espectral do sistema de 11.71875 Hz por bin, a frequência física de 1000 Hz mapeia-se num índice fracionário no domínio da frequência:

$$B_{1000} = \frac{1000}{11.71875} = 85.33 \quad (2)$$

Devido a este índice não inteiro e ao espalhamento de energia condicionado pela janela de Hann, a energia do sinal concentra-se predominantemente nos índices discretos adjacentes: o bin 85, correspondente a 996.09 Hz, e o bin 86, correspondente a 1007.81 Hz.

O algoritmo implementado distribui estes índices por bandas de largura logarítmica. A análise dos limites de integração revela que a banda 72 compreende o intervalo fracionário entre 83.78 e 88.44. No domínio inteiro, o ciclo iterativo encarregue de processar a banda 72 efetua um varrimento desde o bin 83 até ao bin 87.

Como ambos os bins principais, 85 e 86, se encontram contidos dentro desta janela de pesquisa, a função de detecção de máximo consegue capturar a componente mais relevante da energia do tom puro e atribui-la principalmente à banda 72.

Na observação prática, contudo, verifica-se uma assimetria energética relevante: a banda 72 apresenta o pico de amplitude, enquanto a banda 73 apresenta aproximadamente 50% dessa magnitude. Este comportamento não decorre de falhas de processamento, mas sim da geometria da segmentação logarítmica face à resolução da FFT.

A fuga espectral, condicionada pela janela de Hann, distribui a energia em torno da frequência fundamental segundo uma curva de atenuação específica. Dado que a distância entre o pico real, situado no índice 85.33, e o bin 88, associado ao início da banda 73, é relativamente pequena, a banda 73 ainda recebe uma contribuição

significativa da energia do tom. Isto justifica o pico secundário observado na banda 73, enquanto a banda 72 mantém o pico principal.

Este resultado demonstra que a visualização obtida está de acordo com o comportamento esperado da FFT, da janela de Hann e do mapeamento logarítmico implementado no código.

b) Tom de 500 Hz

O índice fracionário do bin para 500 Hz é:

$$B_{500} = \frac{500}{11.71875} = 42.66 \quad (3)$$

A energia do sinal fica dividida principalmente entre o bin 42, correspondente a 492.18 Hz, e o bin 43, correspondente a 503.90 Hz.

A banda 59 começa no bin fracionário 41.47 e a banda 60 começa no bin fracionário 43.78. No domínio inteiro, a banda 59 percorre os bins 41 e 42, enquanto a banda 60 percorre os bins 43, 44 e 45.

Desta forma, a banda 59 deteta a magnitude associada ao bin 42, enquanto a banda 60 deteta a magnitude associada ao bin 43. A energia do tom divide-se, portanto, entre o último bin processado pela banda 59 e o primeiro bin processado pela banda 60. Isto resulta na ativação de duas barras com magnitudes muito próximas.

A observação prática confirma a previsão teórica: as duas barras, correspondentes às bandas 59 e 60, apresentam amplitudes praticamente iguais. Este comportamento decorre do posicionamento da frequência de 500 Hz praticamente sobre a fronteira entre as duas bandas logarítmicas.

As bandas adjacentes, nomeadamente as bandas 58 e 61, recebem contribuições mínimas, correspondentes aos lóbulos laterais mais atenuados da janela de Hann, sendo praticamente invisíveis na visualização.

Estes resultados confirmam a correta aplicação da janela de Hann e o funcionamento adequado do algoritmo de mapeamento logarítmico, demonstrando que a cadeia de processamento está alinhada com a teoria da FFT.

C. Teste Comparativo com Áudio Complexo

Para uma validação final da cadeia de processamento digital de sinal, foi utilizado um sinal de áudio complexo, correspondente a música real. Ao contrário dos testes com tons puros, este tipo de sinal contém várias componentes frequenciais em simultâneo, com amplitudes variáveis ao longo do tempo. Por isso, permite avaliar de forma mais realista o comportamento do sistema perante sinais próximos de uma utilização normal [13], [15].

O espectro gerado pelo STM32, após o processamento da FFT e o mapeamento para bandas logarítmicas, foi renderizado no dashboard Node-RED. Este resultado foi comparado com o espectro apresentado pelo EasyEffects, uma ferramenta de processamento e análise de áudio utilizada em sistemas Linux.

A comparação visual entre os dois analisadores mostra uma elevada semelhança na distribuição das componentes frequenciais. Apesar de não se esperar uma correspondência

perfeitamente idêntica, devido a diferenças no método de suavização, escala gráfica, resolução espectral, normalização e forma de agregação das bandas, a morfologia geral do espectro é bastante semelhante.

Verifica-se que as zonas de maior energia no EasyEffects também surgem destacadas no dashboard Node-RED alimentado pelo STM32. Do mesmo modo, as regiões de menor energia apresentam uma atenuação coerente em ambos os sistemas. Esta paridade visual indica que o processamento realizado pelo STM32 representa corretamente a estrutura espectral do sinal de áudio.



Fig. 4. Comparação entre o espectro apresentado pelo EasyEffects, em cima, e o espectro obtido no Dashboard Node-RED a partir do STM32, em baixo.

Este teste confirma que a implementação da FFT, da janela de Hann e do mapeamento logarítmico das bandas funciona de forma adequada não só para tons puros, mas também para sinais de áudio complexos e dinâmicos.

V. DISCUSSÃO E LIMITAÇÕES

Apesar do desempenho globalmente positivo, o projeto expôs algumas limitações intrínsecas ao hardware e ao protocolo escolhidos, bem como desafios significativos de implementação que merecem ser discutidos.

A. Complexidade da Stack USB e Integração com o DSP

Um dos maiores desafios do projeto foi a compreensão e adaptação da stack USB Audio Class 1.0 da STMicroelectronics. O middleware USB, embora funcional, foi originalmente projetado para um fluxo linear, no qual o áudio é recebido do host e enviado diretamente para o DAC [3], [8].

A introdução de uma camada de processamento DSP entre a recepção e a saída de áudio exigiu uma compreensão aprofundada dos mecanismos internos da stack. Em particular, foi necessário compreender a gestão dos buffers, dos ponteiros internos e da sincronização entre o fluxo USB e a reprodução através do SAI.

A solução encontrada consistiu em contornar a camada de saída original do middleware e alimentar o DMA do SAI a partir de um buffer separado. Esta abordagem exigiu várias iterações até que todos os ponteiros e buffers ficassem corretamente sincronizados. Qualquer erro na gestão dos índices de escrita ou leitura resultava em underruns ou overruns no DAC, provocando falhas no fluxo de áudio.

B. Desafios na Sincronização e Estabilidade do Sistema

A sincronização entre os dois núcleos do STM32H7 e a manutenção de um fluxo de áudio estável revelaram-se tarefas particularmente exigentes. O mecanismo de feedback assíncrono, embora teoricamente bem definido na especificação USB, exigiu um ajuste fino para funcionar corretamente em todas as condições de operação [8].

O resultado final foi um sistema estável, com uma latência aproximada entre 60 ms e 90 ms e sem artefactos audíveis relevantes. Este resultado foi alcançado após várias iterações, demonstrando que a arquitetura dual-core é viável para este tipo de aplicação, embora exija um esforço de desenvolvimento considerável [2].

C. Limitações do Hardware e do Protocolo

1. Protocolo USB Full-Speed

A utilização de USB Full-Speed, associada ao USB Audio Class 1.0, limita a taxa do barramento a 12 Mbps. Como consequência, o sistema fica restringido a áudio de 48 kHz e 24 bits com processamento DSP ativo, não permitindo explorar totalmente o potencial de processamento do STM32H7 [7], [8].

Do ponto de vista computacional, o processador teria capacidade para manipular resoluções superiores, como 96 kHz ou 192 kHz. No entanto, a limitação imposta pelo barramento USB impede a utilização prática dessas taxas neste projeto. Esta limitação também contribui para a latência estimada, uma vez que o intervalo de 1 ms entre frames USB obriga à utilização de buffers suficientemente grandes para absorver o jitter e manter a reprodução estável.

2. Complexidade da Coerência de Memória

O projeto evidenciou também a complexidade associada à arquitetura dual-core do STM32H7, especialmente no que diz respeito à coerência de memória. A existência de cache no Cortex-M7 pode originar inconsistências quando os dados são partilhados entre os dois núcleos ou entre o processador e periféricos com acesso direto à memória, como o DMA [1], [2].

A configuração da MPU, declarando a região SRAM4 como Non-Cacheable, permitiu resolver a comunicação entre os núcleos de forma estável. No entanto, o projeto demonstrou que qualquer descuido no tratamento paralelo dos buffers de DMA pode resultar em corrupção de dados ou em falhas na reprodução de áudio. Assim, a gestão correta da memória revelou-se um ponto crítico para o funcionamento fiável do sistema [1], [3].

3. Latência do Sistema

Embora a latência estimada, aproximadamente entre 60 ms e 90 ms, seja aceitável para reprodução musical, esta limita a utilização do dispositivo em cenários que exijam monitorização em tempo real. Exemplos deste tipo de aplicação incluem gravação de estúdio, instrumentos musicais digitais ou jogos, onde são normalmente desejáveis latências inferiores a 10 ms.

Desta forma, apesar de o sistema apresentar um comportamento estável e adequado para reprodução e processamento de áudio, a latência obtida constitui uma

limitação importante para aplicações interativas ou profissionais de baixa latência.

VI. CONCLUSÃO E TRABALHOS FUTUROS

O projeto demonstrou o potencial do STM32H7 para aplicações de processamento digital de áudio em tempo real. A separação de tarefas entre os dois núcleos revelou-se uma solução eficaz: o Cortex-M7 ficou responsável pela gestão USB, pelo fluxo principal de áudio e pela aplicação dos filtros IIR, enquanto o Cortex-M4 ficou encarregue do processamento matemático associado à FFT e da transmissão dos dados para a plataforma Node-RED [1], [2], [16].

Esta arquitetura permitiu ultrapassar algumas limitações do hardware, mantendo um fluxo de áudio estável, sincronizado de forma assíncrona e com boa qualidade sonora. A utilização da MPU foi também essencial para garantir uma comunicação segura entre os núcleos, reduzindo problemas associados à partilha de memória e à coerência de cache [1], [2].

De forma geral, o sistema final demonstrou ser funcional, estável e adequado ao objetivo proposto, validando a utilização de uma arquitetura dual-core para processamento de áudio com visualização espectral em tempo real.

Para trabalhos futuros e possíveis melhorias, propõem-se as seguintes evoluções.

A. Interface Gráfica Bidirecional com Controlo do Equalizador

Atualmente, a comutação entre perfis de equalização é realizada através de um botão físico. Como melhoria, propõe-se o desenvolvimento de uma interface gráfica bidirecional no Node-RED, permitindo ao utilizador selecionar perfis diretamente no dashboard ou desenhar manualmente a curva de equalização pretendida [16].

Nesta abordagem, os comandos seriam enviados do Node-RED para o STM32 através da mesma USART3 utilizada para a transmissão dos dados da FFT ou, em alternativa, através de uma USART dedicada. Para isso, poderia ser utilizado um protocolo simples baseado em comandos ASCII. O Cortex-M7 interpretaria esses comandos e recalcularia os coeficientes dos filtros Biquad em tempo real, tornando desnecessário o botão físico e reduzindo a dependência de perfis pré-gravados.

Em alternativa, para uma integração mais robusta, poderia ser implementado um endpoint de controlo USB personalizado, recorrendo a pedidos vendor-specific ou class-specific da USB Audio Class. Esta solução exigiria a existência de um daemon no computador, responsável por traduzir os comandos do Node-RED para pedidos USB enviados ao dispositivo. Embora esta abordagem fosse mais complexa, permitiria preservar a USART exclusivamente para a transmissão dos dados da FFT e ofereceria maior flexibilidade ao sistema.

B. Migração para UAC 2.0 High-Speed

A migração para o protocolo USB Audio Class 2.0 representa uma evolução natural do projeto, uma vez que permitiria ultrapassar várias limitações associadas ao USB Audio Class 1.0 e ao barramento USB Full-Speed [8], [9].

1. Aumento da largura de banda

O suporte a USB High-Speed, com uma taxa de 480 Mbps, permitiria trabalhar com frequências de amostragem superiores, como 96 kHz ou 192 kHz, e profundidades de 24 ou 32 bits [7], [9]. Isto permitiria explorar melhor o potencial de processamento do STM32H7 e aproximar o sistema de uma solução de áudio de elevada resolução.

2. Redução da latência

O USB Audio Class 2.0 pode beneficiar da operação em USB High-Speed, onde são utilizados microframes de 125 μ s, em vez dos frames de 1 ms característicos do USB Full-Speed [7], [9]. Esta redução do intervalo base de transmissão permitiria utilizar buffers de pré-preenchimento mais pequenos, reduzindo teoricamente a latência total do sistema. Como resultado, o dispositivo poderia tornar-se mais adequado para cenários que exigem menor latência, como monitorização em estúdio, instrumentos digitais ou jogos.

3. Mecanismo de feedback mais preciso

O mecanismo de feedback no UAC 2.0 permite um controlo de fluxo mais adequado a taxas de amostragem superiores e a arquiteturas de áudio mais exigentes [9]. Isto contribuiria para uma reprodução mais eficiente e para uma melhor sincronização entre o host e o dispositivo USB.

4. Suporte multicanal

Outra vantagem do UAC 2.0 é o suporte a arquiteturas de áudio mais flexíveis, incluindo configurações multicanal [9]. Esta evolução permitiria futuras expansões para sistemas surround ou interfaces de áudio profissionais com vários canais de entrada e saída.

Para concretizar esta migração, seria necessário integrar um PHY USB externo com suporte a High-Speed, uma vez que o STM32H755ZI-Q dispõe do controlador USB, mas não do PHY necessário para operar a esta velocidade [5]. Além disso, seria necessário adaptar ou substituir o middleware USB utilizado, recorrendo a uma implementação compatível com UAC 2.0.

C. Plataforma Alternativa: ESP32-S3 com Bluetooth

Como alternativa ao STM32H7, poderia ser explorada a utilização de um ESP32-S3 em trabalhos futuros. Este SoC da Espressif apresenta algumas características interessantes para aplicações de áudio e conectividade, nomeadamente Bluetooth, Wi-Fi integrado e suporte a USB nativo.

1. Conectividade Bluetooth

A conectividade Bluetooth permitiria a receção de áudio sem fios, eliminando a necessidade de ligação USB para fontes móveis, como telemóveis, tablets ou computadores portáteis. Esta funcionalidade aumentaria a flexibilidade do sistema e aproximá-lo-ia de soluções de áudio mais portáteis.

2. USB nativo

O ESP32-S3 dispõe de controlador USB OTG nativo, embora limitado a Full-Speed. Esta característica permitiria manter alguma compatibilidade com arquiteturas baseadas

em USB, embora com limitações semelhantes às encontradas neste projeto relativamente à largura de banda.

3. Wi-Fi integrado

A presença de Wi-Fi integrado permitiria transmitir os dados do analisador espectral diretamente para um dashboard Web através de WebSocket ou HTTP. Desta forma, seria possível eliminar a necessidade de uma ligação USART ao Node-RED, tornando a visualização mais flexível e independente da ligação física ao computador.

Apesar destas vantagens, a mudança para o ESP32-S3 também apresenta algumas desvantagens. Em primeiro lugar, o ecossistema ESP-IDF é diferente do STM32Cube, o que exigiria uma reescrita significativa do firmware. Além disso, a utilização de áudio por Bluetooth introduziria latências adicionais, podendo comprometer a experiência em cenários interativos ou de baixa latência.

Uma solução intermédia passaria por manter o STM32H7 responsável pela cadeia de áudio crítica, incluindo USB, DSP e DAC, e adicionar um módulo Bluetooth dedicado para receção de áudio sem fios. O sinal poderia ser encaminhado para o STM32 através de uma interface I2S ou PCM. Desta forma, seria possível combinar a potência de processamento do STM32H7 com a conectividade sem fios de um módulo externo, beneficiando do melhor dos dois mundos.

Em síntese, o projeto atingiu os objetivos propostos e demonstrou que a plataforma STM32H7 é adequada para aplicações de áudio digital em tempo real, deixando margem para futuras evoluções ao nível da latência, da resolução de áudio, da interface de controlo e da conectividade sem fios.

REFERÊNCIAS

- [1] STMicroelectronics, *STM32H745/755 and STM32H747/757 advanced Arm®-based 32-bit MCUs reference manual (RM0399)*, 2021. [Online]. Available: <https://www.st.com/en/microcontrollers-microprocessors/stm32h745-755/documentation.html>
- [2] STMicroelectronics, *STM32H745/755 and STM32H747/757 lines dual-core architecture (AN5557)*, 2020. [Online]. Available: https://www.st.com/resource/en/application_note/an5557-stm32h745755-and-stm32h747757-lines-dualcore-architecture-stmicroelectronics.pdf
- [3] STMicroelectronics, *Description of STM32H7 HAL and low-layer drivers (UM2217)*, 2022. [Online]. Available: https://www.st.com/resource/en/user_manual/um2217-description-of-stm32h7-hal-and-lowlayer-drivers-stmicroelectronics.pdf
- [4] STMicroelectronics, *STM32H7 Nucleo-144 boards (MB1363) user manual (UM2408)*, 2025. [Online]. Available: https://www.st.com/resource/en/user_manual/um2408-stm32h7-nucleo144-boards-mb1363-stmicroelectronics.pdf
- [5] STMicroelectronics, *STM32H755xI datasheet*, 2026. [Online]. Available: <https://www.st.com/resource/en/datasheet/stm32h755bi.pdf>
- [6] USB Implementers Forum, *Universal Serial Bus Specification*, Revision 1.1, Sept. 1998. [Online]. Available: <https://esd.cs.ucr.edu/webres/usb11.pdf>
- [7] USB Implementers Forum, *Universal Serial Bus Specification*, Revision 2.0, Apr. 2000. [Online]. Available: <https://www.usb.org/document-library/usb-20-specification>
- [8] USB Implementers Forum, *USB Device Class Definition for Audio Devices*, Release 1.0, Mar. 1998. [Online]. Available: <https://www.usb.org/sites/default/files/audio10.pdf>
- [9] USB Implementers Forum, *USB Device Class Definition for Audio Devices*, Release 2.0, May 2006. [Online]. Available: <https://www.usb.org/document-library/audio-devices-rev-20-and-adapters-agreement>
- [10] USB Implementers Forum, *USB Device Class Definition for Audio Data Formats*, Release 1.0, Mar. 1998. [Online]. Available: <https://www.usb.org/sites/default/files/frmts10.pdf>
- [11] ARM Limited, *CMSIS-DSP software library: Overview*. [Online]. Available: <https://arm-software.github.io/CMSIS-DSP/main/>
- [12] ARM Limited, *Biquad cascade IIR filters using direct form I structure*. [Online]. Available: https://arm-software.github.io/CMSIS-DSP/main/group_BiquadCascadeDF1.htm
- [13] ARM Limited, *Real FFT functions*. [Online]. Available: https://arm-software.github.io/CMSIS-DSP/main/group_RealFFT.html
- [14] R. Bristow-Johnson, "Cookbook formulae for audio EQ biquad filter coefficients." [Online]. Available: <https://webaudio.github.io/Audio-EQ-Cookbook/audio-eq-cookbook.html>
- [15] National Instruments, "Understanding FFTs and windowing." [Online]. Available: <https://www.ni.com/en/shop/data-acquisition/measurement-fundamentals/analog-fundamentals/understanding-ffts-and-windowing.html>
- [16] Node-RED, *Node-RED user guide and general documentation*. [Online]. Available: <https://nodered.org/docs/>
- [17] Texas Instruments, *PCM5102 Datasheet* SLAS764B, May 2011, Revised September 2012. [Online]. Available: <https://www.ti.com/lit/ds/symlink/pcm5102.pdf>